

CSS Cheat Sheet for Your Portfolio (Beginner to Practical)

Use this as a study guide and quick reference while editing your site.

How to read this cheat sheet

- [IMPORTANT] Each section has a concept, when to use it, and an example.
- [IMPORTANT] Copy examples into your CSS and tweak values to learn faster.
- [TIP] Change one value at a time, refresh, and compare the result.

1) Selectors (How CSS chooses elements)

Core selector types

- Element selector: targets all elements of that tag.
 - Example: `p { color: white; }`
- Class selector: reusable style for many elements.
 - Example: `.header-button { border-radius: 6px; }`
- ID selector: one unique element only.
 - Example: `#title-name { font-weight: bold; }`

[IMPORTANT] Prefer classes for reusable styles. Avoid duplicate IDs.

Selector combinations

- Descendant selector: `.header-icons a` means links inside `.header-icons`.
- Multiple selectors: `.card, .panel { padding: 16px; }`

Example:

```
““css
.card h2 {
  font-size: 24px;
}

.card,
.panel {
  border: 1px solid rgb(68, 152, 114);
}
““
```

2) The Box Model (Why spacing looks the way it does)

Every element is:

1. Content
2. Padding (inside space)
3. Border

4. Margin (outside space)

[IMPORTANT] box-sizing: border-box; makes width/height easier to reason about.

Example:

```
““css
* {
  box-sizing: border-box;
}

.card {
  width: 320px;
  padding: 16px;
  border: 2px solid rgb(68, 152, 114);
  margin: 12px;
}
““
```

3) Sizing Units (px, %, rem, vw)

- px: fixed size (good for borders, icon sizes).
- %: relative to parent size.
- rem: relative to root font size (great for scalable typography).
- vw / vh: relative to viewport.

Example:

```
““css
.title {
  font-size: 1.5rem;
}

.container {
  width: 90%;
  max-width: 1200px;
}

.hero {
  min-height: 60vh;
}
““
```

4) Flexbox (Main tool for responsive rows/columns)

Why Flexbox

Use Flexbox when aligning items in one direction (row or column).

Important properties

- display: flex
- justify-content: horizontal distribution in row mode
- align-items: vertical alignment in row mode
- flex-wrap: wrap: allows next line when space is tight
- gap: clean spacing between items

Example (desktop row that wraps on mobile):

```
“css
.main-row {
  display: flex;
  justify-content: center;
  align-items: stretch;
  flex-wrap: wrap;
  gap: 24px;
}
```

Understanding flex: 1 1 320px

- First 1: can grow
- Second 1: can shrink
- 320px: preferred base width

Example card sizing:

```
“css
.card-large {
  flex: 1 1 620px;
}

.card-small {
  flex: 1 1 320px;
}
```

5) Centering Layout on Large Screens

[IMPORTANT] This is the pattern you used in your portfolio.

```
“css
main {
  max-width: 1200px;
  margin: 0 auto;
  padding: 0 20px;
}
```

Why it works:

- max-width stops over-stretching on very wide monitors.
- margin: 0 auto centers the block.

6) Responsive Design with Media Queries

Use breakpoints to adapt layout by screen size.

Example:

```
““css
/ Tablet /
@media (max-width: 768px) {
  .header-title {
    flex-direction: column;
    align-items: center;
  }
}

/ Phone /
@media (max-width: 480px) {
  .header-button {
    min-width: 120px;
    font-size: 14px;
  }
}
““
```

[IMPORTANT] Start desktop-first or mobile-first, but be consistent.

7) Height Behavior: Fixed vs Auto

- Fixed height: stable box size but can clip content.
- height: auto: content decides height (best for phones most of the time).

Example:

```
““css
.card {
  height: 600px;
}

@media (max-width: 768px) {
  .card {
    height: auto;
  }
}
```

```
}  
""
```

[IMPORTANT] Use fixed heights carefully for text-heavy sections.

8) Typography Basics

Useful properties:

- font-family
- font-size
- font-weight
- line-height
- letter-spacing
- text-align

Example:

```
""css  
.body-copy {  
  font-family: RobotoCondensed, Arial, sans-serif;  
  font-size: 16px;  
  line-height: 1.6;  
  color: whitesmoke;  
}  
""
```

[TIP] For readable paragraphs, line-height: 1.5 to 1.8 is usually comfortable.

9) Colors and Reuse with CSS Variables

Instead of repeating color values, define variables once.

Example:

```
""css  
:root {  
  --bg: rgb(36, 36, 36);  
  --accent: rgb(68, 152, 114);  
  --text-soft: rgb(187, 205, 197);  
}  
  
body {  
  background: var(--bg);  
}  
  
.title {  
  color: var(--text-soft);  
}
```

```
}  
""
```

[IMPORTANT] Variables make theme updates much faster.

10) Hover Effects and Transitions

Transitions make property changes smooth.

Example:

```
""css  
.header-button {  
  transition: all 0.3s ease;  
}  
  
.header-button:hover {  
  background-color: rgb(68, 152, 114);  
  color: rgb(36, 36, 36);  
}  
""
```

[TIP] Keep transitions short (0.2s to 0.35s) for responsive feel.

11) Keyframes Animation (Page Fade-In)

```
""css  
body {  
  opacity: 0;  
  animation: pageFadeIn 0.7s ease-out forwards;  
}  
  
@keyframes pageFadeIn {  
  from { opacity: 0; }  
  to { opacity: 1; }  
}  
""
```

forwards keeps final state (opacity: 1) after animation ends.

12) Accessibility: Reduced Motion

[IMPORTANT] Always provide motion fallback.

```
""css  
@media (prefers-reduced-motion: reduce) {
```

```
body {
  animation: none;
  opacity: 1;
}

.header-button,
.header-icon {
  transition: none;
}
}
""
```

13) Common Mistakes and Fixes

1. Too much empty space on one side
 - Check: max-width, margin: 0 auto, justify-content
2. Layout breaks on phone
 - Check: min-width, fixed height, missing flex-wrap
3. Buttons/icons far apart when stacked
 - Check: flex-grow, gap, width rules
4. Hover effect feels jumpy
 - Add/adjust transition

14) Mini Practice Tasks (Hands-on)

Try these in your own CSS:

Task A: Make layout wider

Change:

```
""css
main { max-width: 1200px; }
""
```

To:

```
""css
main { max-width: 1320px; }
""
```

Observe: more horizontal room on desktop.

Task B: Tighten card spacing

Change:

```
""css
```

```
.main-first,  
.main-second,  
.main-third { gap: 24px; }  
""
```

To:

```
""css  
.main-first,  
.main-second,  
.main-third { gap: 16px; }  
""
```

Observe: cards sit closer together.

Task C: Slower page fade

Change:

```
""css  
animation: pageFadeIn 0.7s ease-out forwards;  
""
```

To:

```
""css  
animation: pageFadeIn 1.1s ease-out forwards;  
""
```

Observe: softer entrance.

15) Fast Troubleshooting Checklist

- [IMPORTANT] Is the element using the right selector (class vs ID)?
- [IMPORTANT] Is there a media query overriding your style?
- [IMPORTANT] Is fixed width/height causing overflow?
- [IMPORTANT] Are flex items allowed to wrap?
- [IMPORTANT] Are margins creating fake alignment issues?

16) Reference Snippets You Can Reuse

Responsive card grid

```
""css  
.row {  
  display: flex;  
  flex-wrap: wrap;  
  gap: 20px;  
}
```

```
}
```

```
.card {  
  flex: 1 1 300px;  
}  
““
```

Center everything in a row

```
““css  
.row-center {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
““
```

Safe image sizing

```
““css  
img {  
  max-width: 100%;  
  height: auto;  
}  
““
```

Final notes

You are now using practical CSS patterns that real production sites use:

- centered containers
- responsive flex layouts
- breakpoint-based refinements
- accessible animation

Keep experimenting by changing one variable at a time and observing the result.